

Applied Software Engineering

-

Module Code: CP60019E

Module Title: Applied Software Engineering

Assignment Title: Assignment 2 - Component Software Development

-

Student Name: Mohammed Adnan Osman

Student ID: 33114153

Course: BSc (Hons) Computer Science

Academic Year: 2025/2026

-

Submission Date: 15 January 2026, 23:59

Module Leader: Dr Nasser Matorianpour

Word Count: 6293 (Including code)

-

Table of Contents

Task 1	3
Implementation	3
Object-Oriented Principles	3
Implemented use case mapping	4
Software Engineering best practices	6
Task 2	7
Object Constraint Language (OCL)	7
Business Logic, Rules and Constraints	8
OCL Invariants and Constraints	8
Project	8
Email	9

Attachment	11
User.....	12
Filter	13
Preconditions and Postconditions	14
Authenticate User.....	14
Select Project	16
Apply Filters	17
View Email Details	18
View Attachment	20
Navigation Expressions.....	21
Enforcing OCL in the Implementation	21
Task 3.....	22
Unit Testing	22
Task 4.....	27
Software maintenance, and Sustainable Software Engineering	27
Explanation and Critical Evaluation of Software Design.....	27
Software Maintenance	28
Sustainable Software Engineering (SSE)	28
Bibliography	29
Appendix.....	30
Subject.java	30
Attachment.java	30
Email.java	31
Filter.java	33
User.java.....	35
Observer.java	37
Employee.java.....	37
ClientUser.java.....	37
Project.java	38

AuthService.java	40
ProjectService.java	41
EmailService.java	42
AttachmentService.java	43
StorageService.java	43
KMSSystem.java	44
KMSDemo.java	47
DomainInvariantTests.java	51

Task 1

Implementation

Develop a software application to implement your chosen use case and UML models. As part of your answer, illustrate how the principles of object orientation are implemented. [30 marks]

This task implements the “Search and View Project Emails” use case for the hk.net Knowledge Management System (KMS). The system allows authorized users to authenticate, select a project channel, search archived emails using filters, view email details and attachments, export results, and optionally subscribe to updates. The implementation follows the UML models produced in Assignment 1.

Object-Oriented Principles

Encapsulation

- Domain classes keep the state private (Email, Attachment, Project, User).
- So, access can only be achieved using public getters/ methods.
- ‘User.getNotifications()’, this returns an unmodifiable list which prevents external modifications to it.

Abstraction

- My KMSSystem provides a simple public interface to the system.
- The KMSSystem class also acts as a Facade pattern, simplifying interaction with multiple internal services and reducing coupling between the client code and the system internals.

- The user interface would not need to know internal service logic as it only calls the system operations.

Inheritance

- 'User' is an abstract with a common parent.
- 'Employee' and 'ClientUser' specialize the user type without duplicating the same fields and behaviors.

Polymorphism

- Both the 'Employee' and 'ClientUser' are used through the User type like authentication and observer updates.

Composition

- The 'Project' contains multiple 'Email'.
- 'Email' contains 0..* 'Attachment'
- This means the emails belong to the project context which emulate my UML design when modelling the relationships.

Implemented use case mapping

1. Have you produced a software product which captures the system requirements for one use case?

All mandatory and optional flows of the selected use case are implemented and executed successfully in the KMSDemo class.

Authenticate User:

-> KMSSystem.authenticateUser(userId, password)

--> AuthService.authenticate(...)

Select Project/Client Channel:

-> KMSSystem.selectProject(projectId)

--> ProjectService.selectProject(...)

Apply Filters:

-> KMSSystem.applyFilters(Filter)

--> EmailService.search(Project, Filter)

View Email Details:

-> KMSSystem.viewEmailDetails(emailId)

--> EmailService.getEmailDetails(...)

View Attachment (extend):

-> KMSSystem.viewAttachment(emailId, attachmentId)

--> AttachmentService.getAttachment(...)

--> Attachment.download()

Save/Export Results (extend):

-> KMSSystem.saveOrExportResults(List<Email>)

--> StorageService.exportCsv(...)

Subscribe to Updates (extend / opt-in or no results):

This was implemented using Observer pattern.

Project – Subject, keeps List<Observer>

User – Observer, update()

The user is attached on project selection - ProjectService.selectProject(...){
project.attach(user); }

Adding a new email will trigger a notification –

-> Project.addEmail(...)

--> notifyObservers("email.added", email)

Evidence works:



```
PS C:\Users\User\OneDrive\CS\JAVA\files>
kpspaceStorage\ab2b2e7206de741095b687e17523bca3\redhat.java\jdt_ws\files_b80902a3\bin' 'KMSDemo'
Auth success? true
Search results: 1
- Budget Update Q3 | Revised figures attached.
Opened: Budget Update Q3 | hasAttachment=true
Downloaded bytes: 16
CSV export:
emailId,subject,sentAt,from,to,hasAttachment
E2,"Budget Update Q3",2025-12-23,cfo@client.com,andrea@hk.com,true

Notifications: [P100:email.added]
PS C:\Users\User\OneDrive\CS\JAVA\files>
```

Figure 1: My KMSDemo run

This shows the login success, 1 search result returned, email opened and an attachment downloaded, CSV exported and an observer notification fired (P100:email.added). This shows the end to end in full use case flow.

Software Engineering best practices

2. Have you exhibited software engineering best practices regarding system decomposition, structure, code consistency, documentation etc.?

Decomposition:

I've written the code to break down the system into clear responsibilities:

Controller:

KMSSystem – This is the single-entry point for the use case. It's done to use public methods which are aligned to the case steps in the system flow:

authenticateUser(userId, password)

selectProject(projectId)

applyFilters(filter)

viewEmailDetails(emailId)

viewAttachment(emailId, attachmentId) - Optional

saveOrExportResults(emails) - Optional

Services:

Service classes encapsulate the application logic and coordinate the interactions between the domain model objects.

- 'AuthService' = This is the authentication and to issue tokens
- 'ProjectService' = To access the project and to subscribe. Also to enforce authorization when selecting a project.
- 'EmailService' = To search emails (filtering] and to get email details.
- 'AttachmentService' = To retrieve attachments from an email when requested.
- 'StorageService' = To export CSV files to satisfy the export extension.

This separation makes the system easier to maintain and test. If authentication logic changes, only AuthService needs modification. This follows the Open/Closed Principle - the system is open for extension but closed for modification.

Domain Model:

The domain model represents the real-world concepts of the system.

- Project = This stores the project data and contains multiple 'Email' objects.
- Email = This stores the subject/ body/ from/ to/ date and contains all the optional 'Attachment' objects.
- Attachment = This represents the downloadable files.
- Filter = This encapsulates the search criteria like keywords, sender, date range, hasAttachmentOnly and applies matching logic.
- User
- Additional subclasses

Pattern Interfaces:

Pattern interfaces are used to implement the Observer design pattern.

- 'Project' implements 'Subject' and maintains a list of observers.
- 'User' implements 'Observer' with update(project, event, payload)
- So, when the user selects a project, they are attached as an observer.
- When a new email is added using 'Project.addEmail()', observers are notified automatically.

Code Consistency

I used the same alignment with the UML model I made in assignment 1 to stay consistent with it. Each class I have had its own main responsibilities. Error handling was handled using exceptions like unauthorized project access or when emails or attachments are missing.

Documentation

I have made comments in my code for where constraints apply like preconditions for OCL. The 'KMSDemo' acts as a runnable script to show the use case. A graphical user interface is not required so this demo sufficiently demonstrates the use case fully.

Task 2

Object Constraint Language (OCL)

Incorporate OCL to represent business logic, rules and constraints for the selected use case. As part of your answer, include the correct syntax for pre and post conditions, navigation, and multiplicities. Produce a list of appropriate business rules and constraints. Produce software to implement the specified OCL statements. [30 marks]

Use Case: Search and View Project Emails (hk.net Knowledge Management System – KMS)

Business Logic, Rules and Constraints

1. A user must be authenticated before accessing any project.
2. A user may only select projects they are authorized to access.
3. A project may contain zero or more emails.
4. An email must always have a subject.
5. An email cannot be dated in the future.
6. An email may contain zero or more attachments.
7. Attachments cannot exist without parent email.
8. Search filters must define a valid date range.
9. Only existing emails can be viewed.
10. Attachments can only be accessed if they belong to the selected email.
11. Subscribed users must be notified when a new email is added to a project.

OCL Invariants and Constraints

Project

context Project

inv validProjectId:

projectId <> null and projectId.size() > 0

inv emailMultiplicity:

emails->size() >= 0

-

Implementation for OCL:

validProjectId: Every project must have a unique identifier for lookups and authorization

emailMultiplicity: A project contains zero or more emails (0..* relationship)

UML Multiplicity: Project [1] --> [0..*] Email (composition)

-

Java Enforcement:


```

J Project.java > ...
1  import java.time.LocalDate;
2  import java.util.ArrayList;
3  import java.util.Collections;
4  import java.util.List;
5
6  public final class Project implements Subject {
7      private final String projectId;
8      private final String title;
9      private final String clientName;
10     private final LocalDate createdAt;
11
12     private final List<Email> emails = new ArrayList<>();
13     private final List<Observer> observers = new ArrayList<>();
14
15     // OCL-like constraints:
16     // inv projectIdNotEmpty: projectId.size() > 0
17     // inv emailsNotNulls: emails->forall(e | e <> null)
18     public Project(String projectId, String title, String clientName, LocalDate createdAt) {
19         if (projectId == null || projectId.isEmpty()) {
20             throw new IllegalArgumentException(s: "Project.projectId must not be empty");
21         }
22         this.projectId = projectId;
23         this.title = (title == null) ? "" : title;
24         this.clientName = (clientName == null) ? "" : clientName;
25         this.createdAt = (createdAt == null) ? LocalDate.now() : createdAt;
26     }

```

Full explanation of the code:

The constructor validates 'projectId' is not null or empty before object creation.

'IllegalArgumentException' prevents invalid Project objects from being instantiated.

The 'private final List<Email> emails = new ArrayList<>();' declaration enforces the 0..* multiplicity using a Java collection.

The 'final' keyword ensures the emails list reference cannot be changed after construction, maintaining invariant integrity

Email

context Email

inv subjectNotEmpty:

subject <> null and subject.size() > 0

inv validSentDate:

sentAt <= Date::now()

inv attachmentMultiplicity:

attachments->size() >= 0

-

Implementation for OCL:

subjectNotEmpty: Emails require subjects for searchability and user understanding.

validSentDate: Ensures temporal consistency - emails cannot be sent in the future (physically impossible)

attachmentMultiplicity: An email contains zero or more attachments (0..* relationship)

UML Multiplicity: Email [1] --> [0..*] Attachment (composition)

-

Java Implementation:

```
J Email.java > Email > subject
1 import java.time.LocalDate;
2 import java.util.ArrayList;
3 import java.util.Collections;
4 import java.util.List;
5
6 public final class Email {
7     private final String emailId;
8     private final String subject;
9     private final String body;
10    private final LocalDate sentAt;
11    private final String from;
12    private final String to;
13    private final List<Attachment> attachments = new ArrayList<>();
14
15
16    public Email(String emailId, String subject, String body,
17                LocalDate sentAt, String from, String to,
18                List<Attachment> attachments) {
19
20        if (subject == null || subject.isEmpty()) {
21            throw new IllegalArgumentException(s: "Email.subject must not be empty");
22        }
23        if (from == null || from.isEmpty()) {
24            throw new IllegalArgumentException(s: "Email.from must not be empty");
25        }
26        if (sentAt == null) {
27            throw new IllegalArgumentException(s: "Email.sentAt must not be null");
28        }
29        if (sentAt.isAfter(LocalDate.now())) {
30            throw new IllegalArgumentException(s: "Email.sentAt cannot be in the future");
31        }
32
33        this.emailId = emailId;
34        this.subject = subject;
35        this.body = (body == null) ? "" : body;
36        this.sentAt = sentAt;
37        this.from = from;
38        this.to = (to == null) ? "" : to;
39
40        if (attachments != null) {
41            this.attachments.addAll(attachments);
42        }
43    }
44 }
```

Full explanation of the code:

The 'subjectNotEmpty' invariant is enforced by checking 'if (subject == null || subject.isEmpty())' and throwing an exception if violated.

The 'validSentDate' invariant is enforced through two checks: first ensuring 'sentAt' is not null, then verifying 'sentAt.isAfter(LocalDate.now())' is false.

Temporal validation prevents logically impossible data like an email cannot be sent before it's created.

The 'attachmentMultiplicity' is enforced by the 'private final List<Attachment> attachments = new ArrayList<>();' field, which allows zero or more attachments.

The constructor conditionally adds attachments only if the parameter is not null, maintaining the 0..* relationship.

Attachment

context Attachment

inv fileNameNotEmpty:

fileName <> null and fileName.size() > 0

inv sizeNonNegative:

sizeMb >= 0

-

Implementation for OCL:

fileNameNotEmpty: Attachments must be identifiable by filename for download and display purposes.

sizeNonNegative: File sizes cannot be negative (physical constraint - files have non-negative sizes).

-

Java Implementation:

```

J Attachment.java > ...
1 public final class Attachment {
2     private final String attachmentId;
3     private final String fileName;
4     private final String fileType;
5     private final double sizeMb;
6
7     public Attachment(String attachmentId, String fileName, String fileType, double sizeMb) {
8         if (fileName == null || fileName.isEmpty()) {
9             throw new IllegalArgumentException(s: "Attachment.fileName must not be empty");
10        }
11        if (sizeMb < 0) {
12            throw new IllegalArgumentException(s: "Attachment.sizeMb must be >= 0");
13        }
14        this.attachmentId = attachmentId;
15        this.fileName = fileName;
16        this.fileType = fileType;
17        this.sizeMb = sizeMb;
18    }
19
20    public String getAttachmentId() { return attachmentId; }
21    public String getFileName() { return fileName; }
22    public String getFileType() { return fileType; }
23    public double getSizeMb() { return sizeMb; }
24
25    public byte[] download() {
26        return ("FILE:" + fileName).getBytes();
27    }
28 }
29

```

Full explanation of the code:

The 'fileNameNotEmpty' invariant is checked immediately in the constructor before any field assignment.

The 'sizeNonNegative' invariant uses the condition 'sizeMb < 0' to catch negative values.

Both invariants use 'IllegalArgumentException' with descriptive messages to provide clear feedback on constraint violations.

The use of 'final' fields ensures these invariants cannot be violated after object creation - once an Attachment is constructed with valid data, that data remains valid for the object's lifetime.

User

context User

inv validUserId:

userId <> null and userId.size() > 0

-

Implementation for OCL:

User ID is the primary identifier and must exist for authentication and authorization.

Without a valid userId, the system cannot distinguish between different users or enforce access control.

-

Java Implementation:

```

1  import java.util.ArrayList;
2  import java.util.Collections;
3  import java.util.List;
4
5  public abstract class User implements Observer {
6      private final String userId;
7      private final String name;
8      private final String email;
9      private final String password;
10     private final String role;
11
12     private final List<String> notifications = new ArrayList<>();
13
14     protected User(String userId, String name, String email, String password, String role) {
15         if (userId == null || userId.isEmpty()) {
16             throw new IllegalArgumentException(s: "User.userId must not be empty");
17         }
18         this.userId = userId;
19         this.name = (name == null) ? "" : name;
20         this.email = (email == null) ? "" : email;
21         this.password = (password == null) ? "" : password;
22         this.role = (role == null) ? "" : role;
23     }
24
25     public String getUserId() { return userId; }
26     public String getRole() { return role; }
27
28     public boolean authenticate(String pwd) {
29         return password.equals(pwd);
30     }
31
32     public List<String> getNotifications() {
33         return Collections.unmodifiableList(notifications);
34     }
35
36     @Override
37     public void update(Project project, String event, Object payload) {
38         notifications.add(project.getProjectId() + " : " + event);
39     }
40 }
41

```

Full explanation of the code:

The constructor is 'protected', meaning User can only be instantiated through its subclasses (Employee and ClientUser).

The 'validUserId' invariant is enforced at the base class level, ensuring all subclasses inherit this validation.

The validation occurs before any field assignment, following the fail-fast principle.

This validation applies to both Employee and ClientUser instances, as they both call 'super(userId, ...)' which triggers this check.

Filter

context Filter

inv validDateRange:

startDate = null or endDate = null or startDate <= endDate

-

Implementation for OCL:

If both dates are specified, the start date must not be after the end date. This prevents impossible searches where the search period starts after it ends.

-

Java Implementation:

```

1  Filter.java > Filter > Filter(String, String, LocalDate, LocalDate, boolean)
2  import java.time.LocalDate;
3
4  public final class Filter {
5      private final String keywords;
6      private final String sender;
7      private final LocalDate dateFrom;
8      private final LocalDate dateTo;
9      private final boolean hasAttachmentOnly;
10
11     // OCL-like constraint:
12     // inv validRange: dateFrom.isEmpty() or dateTo.isEmpty() or dateFrom <= dateTo
13     public Filter(String keywords, String sender,
14                   LocalDate dateFrom, LocalDate dateTo,
15                   boolean hasAttachmentOnly) {
16         if (dateFrom != null && dateTo != null && dateFrom.isAfter(dateTo)) {
17             throw new IllegalArgumentException(s: "Filter.dateFrom must be <= dateTo");
18         }
19         this.keywords = keywords;
20         this.sender = sender;
21         this.dateFrom = dateFrom;
22         this.dateTo = dateTo;
23         this.hasAttachmentOnly = hasAttachmentOnly;
24     }
25
26     public boolean matches>Email e {
27         boolean textOk = true;
28         if (keywords != null && !keywords.isEmpty()) {
29             String kw = keywords.toLowerCase();
30             textOk = e.getSubject().toLowerCase().contains(kw)
31                 || e.getBody().toLowerCase().contains(kw);
32         }
33
34         boolean senderOk = true;
35         if (sender != null && !sender.isEmpty()) {
36             senderOk = e.getFrom().toLowerCase().contains(sender.toLowerCase());
37         }
38
39         boolean dateOk = true;
40         if (dateFrom != null && e.getSentAt().isBefore(dateFrom)) dateOk = false;
41         if (dateTo != null && e.getSentAt().isAfter(dateTo)) dateOk = false;
42
43         boolean attachOk = !hasAttachmentOnly || e.hasAttachment();
44
45         return textOk && senderOk && dateOk && attachOk;
46     }
47 }
48

```

Full explanation of the code:

The constraint follows the OCL logic exactly: only validate if both 'dateFrom' and 'dateTo' are non-null

The condition 'dateFrom.isAfter(dateTo)' directly checks if the constraint 'dateFrom <= dateTo' is violated

This allows flexible filter creation: users can specify no dates, only a start date, only an end date, or both dates (as long as the range is valid)

The 'matches()' method applies the filter logic to emails, using the validated date range to check if an email falls within the search period

Preconditions and Postconditions

Authenticate User

context KMSSystem::authenticateUser(userId : String, password : String)

pre userExists:

User.allInstances()->exists(u | u.userId = userId)

post authenticationResult:

result = true implies currentUser <> null

-

Implementation for OCL:

Precondition: The user ID must exist in the system before authentication can succeed

Postcondition: If authentication succeeds (returns true), the current user must be set; if it fails, currentUser remains null

-

Java Implementation:

```

26 // UC: Authenticate User
27 public boolean authenticateUser(String userId, String password) {
28     token = authService.authenticate(userId, password);
29     currentUser = (token == null) ? null : authService.findUserByToken(token);
30     return currentUser != null;
31 }

```

Figure 2: From KMSSystem.java

```

12 public String authenticate(String userId, String password) {
13     User u = users.get(userId);
14     if (u != null && u.authenticate(password)) {
15         String token = "T-" + userId + "-" + System.currentTimeMillis();
16         activeTokens.put(token, userId);
17         return token;
18     }
19     return null;
20 }
21
22 public User findUserByToken(String token) {
23     String userId = activeTokens.get(token);
24     return (userId == null) ? null : users.get(userId);
25 }
26 }
27

```

Figure 3: From AuthService.java

Full explanation of the code:

The precondition 'userExists' is checked by 'users.get(userId)' in AuthService - if the user doesn't exist, it returns null.

The postcondition is enforced by the ternary operator: 'currentUser = (token == null) ? null : authService.findUserByToken(token)'.

If 'token == null' (authentication failed), then 'currentUser' is set to null, and the method returns false.

If 'token != null' (authentication succeeded), then 'currentUser' is set to the found user, and the method returns true.

This ensures: 'return value = true' if and only if 'currentUser != null'.

Select Project

context KMSSystem::selectProject(projectId : String)

pre userAuthenticated:

currentUser <> null

pre authorisedProject:

availableProjects->exists(p | p.projectId = projectId)

post projectSelected:

currentProject.projectId = projectId

-

Implementation for OCL:

Precondition 1: User must be authenticated before selecting projects.

Precondition 2: User must have authorization to access the requested project.

Postcondition: The current project is set to the selected project.

-

Java Implementation:

```
33 // UC: Select Project/Client Channel
34 public Project selectProject(String projectId) {
35     requireUser();
36     currentProject = projectService.selectProject(currentUser, projectId);
37     return currentProject;
38 }
```

Figure 4: From KMSSystem.java


```

63     private void requireUser() {
64         if (currentUser == null) throw new SecurityException(s: "Not authenticated");
65     }

```

Figure 5: From KMSSystem.java

```

12     public List<Project> getProjects(User user) {
13         return projectsByUser.getOrDefault(user.getUserId(), Collections.emptyList());
14     }
15
16     public Project selectProject(User user, String projectId) {
17         return getProjects(user).stream()
18             .filter(p -> p.getProjectId().equals(projectId))
19             .findFirst()
20             .map(p -> { p.attach(user); return p; }) // subscribe for updates
21             .orElseThrow(() -> new SecurityException(s: "User not authorised for project"));
22     }
23 }
24

```

Figure 6: From ProjectService.java

Full explanation of the code:

Precondition 1 'userAuthenticated' is checked by the 'requireUser()' helper method, which throws 'SecurityException' if 'currentUser == null'.

Precondition 2 'authorisedProject' is checked by ProjectService' it only searches through 'getProjects(user)', which returns only the projects the user is authorized to access

The 'orElseThrow()' ensures that if the projectId doesn't exist in the user's authorized projects, a 'SecurityException' is thrown

The postcondition is guaranteed because if the method returns successfully (doesn't throw), then 'currentProject' has been set to the selected project

Additional behavior: 'map(p -> { p.attach(user); return p; })' automatically subscribes the user as an observer, implementing the "Subscribe to Updates" use case extension

Apply Filters

context KMSSystem::applyFilters(filter : Filter)

pre projectSelected:

currentProject <> null

post validResults:

result->forAll(e | e.ocllsKindOf(Email))

-

Implementation for OCL:

Precondition: A project must be selected before filtering emails.

Postcondition: All results are Email objects (type checking).

-

Java Implementation:

```

40 // UC: Apply Filters
41 public List<Email> applyFilters(Filter filter) {
42     requireProject();
43     return emailService.search(currentProject, filter);
44 }

```

Figure 7: From KMSSystem.java

```

67 private void requireProject() {
68     if (currentProject == null) throw new IllegalStateException(s: "No project selected");
69 }
70 }
71

```

Figure 8: From KMSSystem.java

```

5 public final class EmailService {
6     public List<Email> search(Project project, Filter filter) {
7         List<Email> out = new ArrayList<>();
8         for (Email e : project.getEmails()) {
9             if (filter.matches(e)) out.add(e);
10        }
11        return out;
12    }

```

Figure 9: From EmailService.java

Full explanation of the code:

The precondition is checked by 'requireProject()', which throws 'IllegalStateException' if no project has been selected

The postcondition is guaranteed by Java's type system: the return type 'List<Email>' ensures at compile-time that only Email objects can be in the result

The EmailService.search() method only adds Email objects to the output list 'out.add(e)' where 'e' is of type Email.

The Filter.matches() method only accepts Email objects as input

This shows how OCL type constraints can be implemented through Java's static type checking, providing compile-time validation

View Email Details

context KMSSystem::viewEmailDetails(emailId : String)

pre emailExists:

currentProject.emails->exists(e | e.emailId = emailId)

post emailReturned:

result.emailId = emailId

-

Implementation for OCL:

Precondition: The email must exist in the current project.

Postcondition: The returned email has the requested ID.

-

Java Implementation:

```

46 // UC: View Email Details
47 public Email viewEmailDetails(String emailId) {
48     requireProject();
49     return emailService.getEmailDetails(currentProject, emailId);
50 }

```

Figure 10: From KMSSystem.java

```

14 public Email getEmailDetails(Project project, String emailId) {
15     return project.getEmails().stream()
16         .filter(e -> e.getEmailId().equals(emailId))
17         .findFirst()
18         .orElseThrow(() -> new NoSuchElementException(s: "Email not found"));
19 }
20 }
21

```

Figure 11: From EmailService.java

Full explanation of the code:

The precondition is checked by the stream operations: ‘filter(e -> e.getEmailId().equals(emailId))’ searches only within ‘currentProject.emails’.

If no email matches the ‘emailId’, ‘findFirst()’ returns an empty ‘Optional’, and ‘orElseThrow()’ throws ‘NoSuchElementException’.

This implements the OCL precondition: the method only succeeds if ‘currentProject.emails->exists(e | e.emailId = emailId)’ is true.

The postcondition is guaranteed by the filter condition: if the method returns successfully, it returns the email where e.getEmailId().equals(emailId), which means result.emailId = emailId.

The use of streams provides a declarative approach that closely matches the OCL navigation expression.

View Attachment

context KMSSystem::viewAttachment(emailId : String, attachmentId : String)

pre attachmentExists:

```
currentProject.emails->exists(e |
    e.emailId = emailId and
    e.attachments->exists(a | a.attachmentId = attachmentId)
)
```

-

Implementation for OCL:

The attachment must exist within an email in the current project. This is a nested navigation constraint spanning two associations.

-

Java Implementation:

```
51
52 // UC extension: View Attachment [email.hasAttachment]
53 public Attachment viewAttachment(String emailId, String attachmentId) {
54     Email e = viewEmailDetails(emailId);
55     return attachmentService.getAttachment(e, attachmentId);
56 }
57
```

Figure 12: From KMSSystem.java

```
AttachmentService.java > ...
1 import java.util.NoSuchElementException;
2
3 public final class AttachmentService {
4     public Attachment getAttachment(Email email, String attachmentId) {
5         return email.getAttachments().stream()
6             .filter(a -> a.getAttachmentId().equals(attachmentId))
7             .findFirst()
8             .orElseThrow(() -> new NoSuchElementException(s: "Attachment not found"));
9     }
10 }
11
```

Figure 13: From AttachmentService.java

Full explanation of the code:

The precondition is checked in two stages, matching the nested OCL structure:

1. 'viewEmailDetails(emailId)' checks that the email exists in the current project
2. 'attachmentService.getAttachment(e, attachmentId)' checks that the attachment exists in that email

If either check fails, an appropriate exception is thrown 'NoSuchElementException'.

This two-stage checking makes the precondition verification more modular and follows the principle of separating concerns

The two-stage check exactly matches the OCL structure: 'currentProject.emails->exists(e | ... and e.attachments->exists(a | ...))'.

Navigation Expressions

A navigation expression is any OCL expression that moves from one object to another through an association. Examples of this include:

currentProject.emails – Simple navigation

currentProject.emails.attachments – Nested navigation

User.allInstances() – System-wide navigation

Enforcing OCL in the Implementation

The OCL constraints are enforced in the Java implementation as follows:

- Invariants are validated during object construction.
- Preconditions are checked before executing system operations.
- Postconditions are ensured through the logic method.
- Multiplicities are enforced using Java collections.
- Observer constraints are enforced using the Observer design pattern.

1. Have produced correct OCL to represent the business logic, rules and constraints for the use case?

Business Rules and Logic List = 11 Rules

Invariants = Project, Email, Attachment, User, Filter

Preconditions = authenticateUser, selectProject, applyFilters, viewEmailDetails, viewAttachment

Postconditions = authenticateUser, selectProject, applyFilters, viewEmailDetails

Implementation Mapping = I have explained the java code and how it enforces invariants, preconditions and postconditions.

2. Have you included the correct syntax for multiplicities?

A few of these associations include:

Association	Multiplicity	UML Notation	OCL Constraint	Java Implementation
Project -> Email	1 to 0..*	Project[1] -> [0..*] Email	emails->size() >= 0	List<Email> emails
Email -> Attachment	1 to 0..*	Email[1] -> [0..*] Attachment	attachments->size() >= 0	List<Attachment> attachments
Project -> Observer	1 to 0..*	Project[1] -> [0..*] Observer	observers->size() >= 0	List<Observer> observers

Task 3

Unit Testing

Produce Software testing, documentation, and verification considerations [20 marks]

1. Have you demonstrated understanding of software testing including appropriate code?

I have implemented 7 Junit 5-unit tests that validate the OCL constraints from Task 2 using exception testing. The code for it is below in the appendix section called:

DomainInvariantTests.java. The 7 total tests consist of: 4 Email tests, 2 Attachment tests and 1 filter test. Each test tries to create an object with invalid data. If the objects get created, then the test fails. On the contrary if the test throws an error, then the test passes. The tests are grouped by class (Email, Attachment, Filter).

I used '@Test' annotation marks for each test method. I used 'assertThrows()' to validate exception handling and used 'assertTrue()' to validate exception messages.

2. Have you managed to implement appropriate testing for the code you have produced and in relation to the specified OCL?

Yes for all 7 test validations OCL constraints I wrote in task 2.

Test #	Test Name	OCL Constraint	What It Validates
1	emailWithNullSubjectShouldFail	subject <> null	Null subject rejected
2	emailWithEmptySubjectShouldFail	subject.size() > 0	Empty subject rejected

3	emailWithFutureDateShouldFail	sentAt <= Date::now()	Future date rejected
4	emailWithNullFromShouldFail	from <> null and from.size() > 0	Null sender rejected
5	attachmentWithNullFileNameShouldFail	fileName <> null and fileName.size() > 0	Null fileName rejected
6	attachmentWithNegativeSizeShouldFail	sizeMb >= 0	Negative size rejected
7	filterWithInvalidDateRangeShouldFail	dateFrom <= dateTo	Invalid date range rejected

Test #1/ #2 – null subject and empty subject

These tests verify that the Email constructor rejects both null and empty string subjects, ensuring every email has meaningful subject text. This is critical for the hk.net KMS because subjects appear in search results and email lists.

Test #3 – future dates

This test verifies emails cannot be dated in the future, which is logically impossible. The test creates a date for tomorrow and confirms the constructor rejects it with an appropriate error message.

Test #4 – null sender

This test verifies that every email must have a sender. Emails without senders are meaningless in a knowledge management context where knowing the source of information is critical.

Test #5 – null filename

This test verifies attachments must have file names. Files without names cannot be properly identified, downloaded, or managed in the system.

Test #6 – negative size

This test verifies file sizes cannot be negative, which is a physical impossibility. The test passes -1.0 as the size and confirms the constructor rejects it.

Test #7 – invalid date range

This test verifies date range filters are logically valid. The test creates a filter where the start date (December 31, 2025) is after the end date (January 1, 2025), which is invalid.

3. Have you provided seed or testing data for the system?

Testing data

Test #1/2: Email Subject Test: - All other parameters valid.

```

19  @Test
20  void emailWithNullSubjectShouldFail() {
21      IllegalArgumentException exception = assertThrows(
22          IllegalArgumentException.class,
23          () -> new Email(
24              "id1",
25              null,           // subject (invalid)
26              "body",
27              LocalDate.now(),
28              "sender",
29              "receiver",
30              List.of()
31          )
32      );
33
34      assertTrue(
35          exception.getMessage().toLowerCase().contains(s: "subject"),
36          "Expected exception message to mention 'subject'"
37      );
38  }

```

Figure 14: Test 1: subject = null (invalid)

```

40  @Test
41  void emailWithEmptySubjectShouldFail() {
42      IllegalArgumentException exception = assertThrows(
43          IllegalArgumentException.class,
44          () -> new Email(
45              "id2",
46              "",           // subject (empty - invalid)
47              "body",
48              LocalDate.now(),
49              "sender",
50              "receiver",
51              List.of()
52          )
53      );
54
55      assertTrue(
56          exception.getMessage().toLowerCase().contains(s: "subject"),
57          "Expected exception message to mention 'subject'"
58      );
59  }
60

```

Figure 15: Test 2: subject = "" (empty, invalid)

Test #3: Email Date Test: – All other parameters valid

```

63  LocalDate tomorrow = LocalDate.now().plusDays(daysToAdd: 1);
64
65  IllegalArgumentException exception = assertThrows(
66      IllegalArgumentException.class,
67      () -> new Email(
68          "id3",
69          "Test Subject",
70          "body",
71          tomorrow,           // sentAt (future - invalid)
72          "sender",
73          "receiver",
74          List.of()
75      )

```

Figure 16: sentAt = tomorrow (future date, invalid)

Test #4: Email Sender Test: - All other parameters valid


```

89         () -> new Email(
90             "id4",
91             "Test Subject",
92             "body",
93             LocalDate.now(),
94             null,           // from (invalid)
95             "receiver",
96             List.of()
97         )

```

Figure 17: from = null (invalid)

Test #5: Attachment FileName Test – Size is 1 MB which is valid.

```

112         () -> new Attachment(
113             "a1",
114             null,           // fileName (invalid)
115             "application/pdf",
116             1.0
117         )

```

Figure 18: fileName = null (invalid)

Test #6: Attachment Size Test – filename valid

```

131         () -> new Attachment(
132             "a2",
133             "file.pdf",
134             "application/pdf",
135             -1.0           // sizeMb (negative - invalid)
136         )

```

Figure 19: sizeMb = -1.0 (negative, invalid)

Test #7: Filter Date Range Test

```

150         LocalDate startDate = LocalDate.of(year: 2025, month: 12, dayOfMonth: 31);
151         LocalDate endDate = LocalDate.of(year: 2025, month: 1, dayOfMonth: 1); // End before start (invalid)
152
153         IllegalArgumentException exception = assertThrows(
154             IllegalArgumentException.class,
155             () -> new Filter(
156                 "keyword",
157                 "sender",
158                 startDate,           // dateFrom
159                 endDate,           // dateTo (before dateFrom - invalid)
160                 false
161             )
162         );

```

System Seed Data

Authentication testing:

```

11         // Users
12         Employee emp = new Employee(userId: "emp1", name: "Andrea", email: "andrea@hk.com", password: "secret", department: "PR");
13         ClientUser client = new ClientUser(userId: "cli1", name: "Client A", email: "client@a.com", password: "secret", clientCompany: "ClientA Ltd");
14

```

There are 2 users with different roles. One is an employee and the other a clientUser.

Project selection testing:

```

15         Map<String, User> users = new HashMap<>();
16         users.put(emp.getUserId(), emp);
17         users.put(client.getUserId(), client);

```

Project-to-user authorization mappings.

Email search testing:

```

27         from: "ted@hk.com",
28         to: "team@hk.com",

```

```

39         from: "cfo@client.com",
40         to: "andrea@hk.com",

```

There are 2 emails with different characteristics.

Attachment Testing:

```

32         Attachment a1 = new Attachment(attachmentId: "A1", fileName: "budget.xlsx", fileType: "application/vnd.ms-excel", sizeMb: 0.3);

```

Email 2 has an attachment, but Email 1 does not.

Filter Testing:

The emails as seen above have different dates, subjects and senders.

4. Do you understand the TDD approach? Did the implementation address all the required functionality for the system?

TDD approach means test-driven development. The steps include writing test (Red Phase) first so test fails as code doesn't exist yet, (Green Phase) then write code to make the test pass and then (Refactor) improve the code to make it better while keeping the test passing and not throwing errors.

The implementation works on all classes built (Email, Attachment and Project), all services being worked (login, search and filters). I completed all the use cases from login to viewing the emails.

5. Have you provided clear and concise instructions regarding how the system and its tests should be launched?

Concise instruction of how I ran the tests:

- 1) Locate where files are saved and extract all
- 2) mvn test
- 3) open DomainInvariantTests.java
- 4) click green play button next to 'void'
- 5) see results at the bottom.

```

[INFO] -----
[INFO] Running com.hkUNET.kms.DomainInvariantTests
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.086 s -- in com.hkUNET.kms.DomainInvariantTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.401 s
[INFO] Finished at: 2026-01-14T02:38:03Z
[INFO] -----

```

6. Do you have an understanding of the testing requirements?

I tested one section at a time. Where the checks check email, attachment and filter separately which make sit faster to run. I used negative testing where the test has wrong inputs and the data uses bad inputs like null, empty or negative so the checks stop for bad data. This helps to catch bugs early on and to make sure the code does what it's designed for. So, in future reference if I test something by changing code the test will tell me immediately if it was unsuccessful.

7. Have you produced code listing and explanation?

Yes, all code explanations are above and all code in appendix section.

Task 4

Software maintenance, and Sustainable Software Engineering

With reference to your software design and implementation, discuss and critically evaluate software maintenance, Sustainable Software Engineering (SSE) practices and principles.

[20 marks]

1. Have you provided an explanation and a critical evaluation of your software design, in terms of software maintenance, Sustainable Software Engineering (SSE) practices and principles.

Explanation and Critical Evaluation of Software Design

The Knowledge Management System is designed using modular architecture that separates domain models, service layers, and the controller. This structure improves maintainability by ensuring that each component has a well-defined responsibility. The main domain includes Project, Email, Attachment, and User. They encapsulate business rules through enforced Object Constraint Language invariants, preventing invalid system states from occurring.

To have UML models, OCL specifications, and Java implementation to align with each other improves traceability and understandability of the system. Constraints defined at the design level are explicitly enforced within the code, reducing ambiguity and simplifying future changes and updates. The OCL I designed improves readability and maintainability for a system. This approach will put responsibility on the developer to ensure constraints remain consistently enforced. In larger or safer critical systems, contract-based tools probably provide stronger guarantees, but for this system I made it understandable not to use those tools.

Software Maintenance

The system supports effective software maintenance through clear separation of concerns and modular design. Domain validation occurs at object construction time so invalid objects can't exist within the system. This reduces the likelihood of defects propagating to later execution stages and lowers the cost of corrective maintenance.

Service classes such as AuthService, ProjectService, and EmailService isolate business logic, enabling adaptive and perfective maintenance without impacting unrelated components. For example, changes to authentication mechanisms or project authorization rules would only require modification within the relevant service, leaving the rest of the system unaffected. This improves system evolvability and reduces regression risk.

Automated testing using JUnit 5 provides regression protection by ensuring that future changes do not violate existing OCL constraints. This supports long-term maintainability by allowing developers to refactor or extend functionality with confidence. Maven dependency management further enhances maintainability by standardizing build and test execution across environments.

Although exception-based validation is effective for this system, it provides limited contextual feedback compared to more advanced contract-based or logging frameworks. Nevertheless, for the current system scale and academic scope, this approach balances robustness, simplicity, and maintainability appropriately.

Sustainable Software Engineering (SSE)

From a technical sustainability perspective, the modular architecture and use of design patterns (Facade and Observer) support long-term system evolution with minimal rework. Efficient filtering logic ensures that only relevant emails are processed, reducing unnecessary computation. The Observer pattern ensures notifications are triggered only when new emails are added, avoiding redundant system activity.

From an economic sustainability perspective, early validation through enforced constraints and automated testing reduces debugging effort and long-term development cost. Preventing invalid data at creation time lowers maintenance overhead and supports faster development cycles. The separation of responsibilities across services further reduces the cost of future enhancements or bug fixes.

From an environmental sustainability perspective, while the system does not explicitly measure energy consumption, its design avoids unnecessary processing by minimizing redundant searches and notifications. Event-driven updates ensure that system resources are used only when required. A limitation of the current system is the lack of explicit energy monitoring. Future work could include performance logging or efficiency analysis under

higher workloads. Despite this, the current design aligns well with environmental SSE principles at this level.

From a social sustainability perspective, the KMS improves collaboration and knowledge sharing between employees and clients by providing structured, searchable access to archived project emails. New team members can quickly review prior communications, reducing onboarding time and improving continuity when staff change. This reduces reliance on informal knowledge transfer and supports fair access to organizational knowledge across teams.

From an individual sustainability perspective, the system design improves developer experience through clear architectural separation, enforced constraints, and readable service interfaces. Fail-fast validation, meaningful exceptions, and automated unit tests reduce cognitive load during development and debugging. This supports developer productivity, reduces frustration, and lowers the likelihood of human error over the system's lifetime.

Bibliography/ References

Fowler M., (2003) UML Distilled: A Brief Guide to the Standard Object Modeling Language, Addison-Wesley Chapters 1 – 4.

Matoorianpour, N. (2025). Introduction to Object Constraint Language OCL (Week 9) [PowerPoint slides]. University of West London.

Matoorianpour, N. (2025). Object Constraint Language Pre Post Conditions (Week 10) [PowerPoint slides]. University of West London.

Matoorianpour, N. (2025). Software Re-engineering (Week 11) [PowerPoint slides]. University of West London.

Matoorianpour, N. (2025). Software Development (Week 12) [PowerPoint slides]. University of West London.

Schmitt, J. (2025) Test-driven development (TDD) explained. URL: <https://circleci.com/blog/test-driven-development-tdd/> [Last Accessed: 13 January 2026].

Stefan, B. Sam, B. Johannes, L. Matthias, M. Marc, P. Christian, S (n.d.) JUnit 5 User Guide URL: <https://docs.junit.org/5.5.0/user-guide/index.html> [Last Accessed: 12 January 2025].

Appendix

Subject.java

```
public interface Subject {  
    void attach(Observer o);  
    void detach(Observer o);  
    void notifyObservers(String event, Object payload);  
}
```

Attachment.java

```
public final class Attachment {  
    private final String attachmentId;  
    private final String fileName;  
    private final String fileType;  
    private final double sizeMb;  
  
    // OCL-like constraints:  
    // inv fileNameNotEmpty: fileName.size() > 0  
    // inv sizeNonNegative: sizeMb >= 0  
    public Attachment(String attachmentId, String fileName, String fileType, double sizeMb) {  
        if (fileName == null || fileName.isEmpty()) {  
            throw new IllegalArgumentException("Attachment.fileName must not be empty");  
        }  
        if (sizeMb < 0) {  
            throw new IllegalArgumentException("Attachment.sizeMb must be >= 0");  
        }  
        this.attachmentId = attachmentId;  
    }  
}
```

```
        this.fileName = fileName;

        this.fileType = fileType;

        this.sizeMb = sizeMb;
    }

    public String getAttachmentId() { return attachmentId; }

    public String getFileName() { return fileName; }

    public String getFileType() { return fileType; }

    public double getSizeMb() { return sizeMb; }

    public byte[] download() {
        return ("FILE:" + fileName).getBytes();
    }
}
```

Email.java

```
import java.time.LocalDate;

import java.util.ArrayList;

import java.util.Collections;

import java.util.List;

public final class Email {

    private final String emailId;

    private final String subject;

    private final String body;

    private final LocalDate sentAt;

    private final String from;
```

```
private final String to;

private final List<Attachment> attachments = new ArrayList<>();

public Email(String emailId, String subject, String body,
             LocalDate sentAt, String from, String to,
             List<Attachment> attachments) {

    if (subject == null || subject.isEmpty()) {
        throw new IllegalArgumentException("Email.subject must not be empty");
    }
    if (from == null || from.isEmpty()) {
        throw new IllegalArgumentException("Email.from must not be empty");
    }
    if (sentAt == null) {
        throw new IllegalArgumentException("Email.sentAt must not be null");
    }
    if (sentAt.isAfter(LocalDate.now())) {
        throw new IllegalArgumentException("Email.sentAt cannot be in the future");
    }

    this.emailId = emailId;
    this.subject = subject;
    this.body = (body == null) ? "" : body;
    this.sentAt = sentAt;
    this.from = from;
    this.to = (to == null) ? "" : to;
```



```
    if (attachments != null) {  
        this.attachments.addAll(attachments);  
    }  
}
```

```
public String getEmailId() { return emailId; }  
public String getSubject() { return subject; }  
public String getBody() { return body; }  
public LocalDate getSentAt() { return sentAt; }  
public String getFrom() { return from; }  
public String getTo() { return to; }
```

```
public boolean hasAttachment() { return !attachments.isEmpty(); }  
public List<Attachment> getAttachments() { return  
Collections.unmodifiableList(attachments); }
```

```
public String preview() {  
    return body.length() > 80 ? body.substring(0, 80) + "...": body;  
}  
}
```

Filter.java

```
import java.time.LocalDate;
```

```
public final class Filter {  
    private final String keywords;
```

```
private final String sender;

private final LocalDate dateFrom;

private final LocalDate dateTo;

private final boolean hasAttachmentOnly;


public Filter(String keywords, String sender,
              LocalDate dateFrom, LocalDate dateTo,
              boolean hasAttachmentOnly) {

    if (dateFrom != null && dateTo != null && dateFrom.isAfter(dateTo)) {
        throw new IllegalArgumentException("Filter.dateFrom must be <= dateTo");
    }

    this.keywords = keywords;
    this.sender = sender;
    this.dateFrom = dateFrom;
    this.dateTo = dateTo;
    this.hasAttachmentOnly = hasAttachmentOnly;
}


public boolean matches(Email e) {
    boolean textOk = true;

    if (keywords != null && !keywords.isEmpty()) {
        String kw = keywords.toLowerCase();

        textOk = e.getSubject().toLowerCase().contains(kw)
            || e.getBody().toLowerCase().contains(kw);
    }
}
```

```
}

    boolean senderOk = true;

    if (sender != null && !sender.isEmpty()) {
        senderOk = e.getFrom().toLowerCase().contains(sender.toLowerCase());
    }

    boolean dateOk = true;

    if (dateFrom != null && e.getSentAt().isBefore(dateFrom)) dateOk = false;
    if (dateTo != null && e.getSentAt().isAfter(dateTo)) dateOk = false;

    boolean attachOk = !hasAttachmentOnly || e.hasAttachment();

    return textOk && senderOk && dateOk && attachOk;
}
}
```

User.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public abstract class User implements Observer {
    private final String userId;
    private final String name;
    private final String email;
    private final String password;
```

```
private final String role;

private final List<String> notifications = new ArrayList<>();


protected User(String userId, String name, String email, String password, String role) {
    if (userId == null || userId.isEmpty()) {
        throw new IllegalArgumentException("User.userId must not be empty");
    }

    this.userId = userId;

    this.name = (name == null) ? "" : name;

    this.email = (email == null) ? "" : email;

    this.password = (password == null) ? "" : password;

    this.role = (role == null) ? "" : role;
}


public String getUserId() { return userId; }

public String getRole() { return role; }


public boolean authenticate(String pwd) {
    return password.equals(pwd);
}


public List<String> getNotifications() {
    return Collections.unmodifiableList(notifications);
}
```

```
@Override
```

```
public void update(Project project, String event, Object payload) {
```

```
    notifications.add(project.getProjectId() + ":" + event);
```

```
}
```

```
}
```

Observer.java

```
public interface Observer {
```

```
    void update(Project project, String event, Object payload);
```

```
}
```

Employee.java

```
public final class Employee extends User {
```

```
    private final String department;
```

```
    public Employee(String userId, String name, String email, String password, String department) {
```

```
        super(userId, name, email, password, "EMPLOYEE");
```

```
        this.department = (department == null) ? "" : department;
```

```
}
```

```
    public String getDepartment() { return department; }
```

```
}
```

ClientUser.java

```
public final class ClientUser extends User {
```

```
    private final String clientCompany;
```

```
public ClientUser(String userId, String name, String email, String password, String
clientCompany) {
    super(userId, name, email, password, "CLIENT");
    this.clientCompany = (clientCompany == null) ? "" : clientCompany;
}

public String getClientCompany() { return clientCompany; }
}
```

Project.java

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public final class Project implements Subject {
    private final String projectId;
    private final String title;
    private final String clientName;
    private final LocalDate createdAt;

    private final List<Email> emails = new ArrayList<>();
    private final List<Observer> observers = new ArrayList<>();

    public Project(String projectId, String title, String clientName, LocalDate createdAt) {
```

```
    if (projectId == null || projectId.isEmpty()) {  
        throw new IllegalArgumentException("Project.projectId must not be empty");  
    }  
    this.projectId = projectId;  
    this.title = (title == null) ? "" : title;  
    this.clientName = (clientName == null) ? "" : clientName;  
    this.createdAt = (createdAt == null) ? LocalDate.now() : createdAt;  
}
```

```
public String getProjectId() { return projectId; }  
public String getTitle() { return title; }  
public String getClientName() { return clientName; }  
public LocalDate getCreatedAt() { return createdAt; }
```

```
public void addEmail(Email e) {  
    if (e == null) throw new IllegalArgumentException("Email must not be null");  
    emails.add(e);  
    notifyObservers("email.added", e);  
}
```

```
public List<Email> getEmails() {  
    return Collections.unmodifiableList(emails);  
}
```

@Override

```
public void attach(Observer o) {
```

```
        if (o != null && !observers.contains(o)) observers.add(o);
    }
```

```
@Override
```

```
public void detach(Observer o) {
    observers.remove(o);
}
```

```
@Override
```

```
public void notifyObservers(String event, Object payload) {
    for (Observer o : new ArrayList<>(observers)) {
        o.update(this, event, payload);
    }
}
}
```

AuthService.java

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public final class AuthService {
    private final Map<String, User> users;
    private final Map<String, String> activeTokens = new HashMap<>();

    public AuthService(Map<String, User> users) {
        this.users = users;
    }
}
```



```
public String authenticate(String userId, String password) {  
    User u = users.get(userId);  
    if (u != null && u.authenticate(password)) {  
        String token = "T-" + userId + "-" + System.currentTimeMillis();  
        activeTokens.put(token, userId);  
        return token;  
    }  
    return null;  
}
```

```
public User findUserByToken(String token) {  
    String userId = activeTokens.get(token);  
    return (userId == null) ? null : users.get(userId);  
}  
}
```

ProjectService.java

```
import java.util.Collections;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
public final class ProjectService {  
    private final Map<String, List<Project>> projectsByUser;  
  
    public ProjectService(Map<String, List<Project>> projectsByUser) {  
        this.projectsByUser = projectsByUser;  
    }  
}
```

```
}
```

```
public List<Project> getProjects(User user) {  
    return projectsByUser.getOrDefault(user.getUserId(), Collections.emptyList());  
}
```

```
public Project selectProject(User user, String projectId) {  
    return getProjects(user).stream()  
        .filter(p -> p.getProjectId().equals(projectId))  
        .findFirst()  
        .map(p -> { p.attach(user); return p; }) // subscribe for updates  
        .orElseThrow(() -> new SecurityException("User not authorised for project"));  
}  
}
```

EmailService.java

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.NoSuchElementException;  
  
public final class EmailService {  
    public List<Email> search(Project project, Filter filter) {  
        List<Email> out = new ArrayList<>();  
        for (Email e : project.getEmails()) {  
            if (filter.matches(e)) out.add(e);  
        }  
        return out;  
    }  
}
```

```
}
```

```
public Email getEmailDetails(Project project, String emailId) {  
    return project.getEmails().stream()  
        .filter(e -> e.getEmailId().equals(emailId))  
        .findFirst()  
        .orElseThrow(() -> new NoSuchElementException("Email not found"));  
}  
}
```

AttachmentService.java

```
import java.util.NoSuchElementException;  
  
public final class AttachmentService {  
    public Attachment getAttachment(Email email, String attachmentId) {  
        return email.getAttachments().stream()  
            .filter(a -> a.getAttachmentId().equals(attachmentId))  
            .findFirst()  
            .orElseThrow(() -> new NoSuchElementException("Attachment not found"));  
    }  
}
```

StorageService.java

```
import java.util.List;  
  
public final class StorageService {  
    public byte[] exportCsv(List<Email> emails) {  
        StringBuilder sb = new StringBuilder();
```

```
sb.append("emailId,subject,sentAt,from,to,hasAttachment\n");
for (Email e : emails) {
    sb.append(e.getEmailId()).append(",\n")
        .append(e.getSubject().replace("\n", "\\n")).append("\n");
    .append(e.getSentAt()).append(",")
    .append(e.getFrom()).append(",")
    .append(e.getTo()).append(",")
    .append(e.hasAttachment()).append("\n");
}
return sb.toString().getBytes();
}
}
```

KMSSystem.java

```
import java.util.List;

public final class KMSSystem {

    private final AuthService authService;
    private final ProjectService projectService;
    private final EmailService emailService;
    private final AttachmentService attachmentService;
    private final StorageService storageService;

    private String token;
    private User currentUser;
    private Project currentProject;
```

```
public KMSSystem(AuthService authService,
    ProjectService projectService,
    EmailService emailService,
    AttachmentService attachmentService,
    StorageService storageService) {
    this.authService = authService;
    this.projectService = projectService;
    this.emailService = emailService;
    this.attachmentService = attachmentService;
    this.storageService = storageService;
}

// UC: Authenticate User
public boolean authenticateUser(String userId, String password) {
    token = authService.authenticate(userId, password);
    currentUser = (token == null) ? null : authService.findUserByToken(token);
    return currentUser != null;
}

// UC: Select Project/Client Channel
public Project selectProject(String projectId) {
    requireUser();
    currentProject = projectService.selectProject(currentUser, projectId);
    return currentProject;
}
```

// UC: Apply Filters

```
public List<Email> applyFilters(Filter filter) {  
    requireProject();  
    return emailService.search(currentProject, filter);  
}
```

// UC: View Email Details

```
public Email viewEmailDetails(String emailId) {  
    requireProject();  
    return emailService.getEmailDetails(currentProject, emailId);  
}
```

// UC extension: View Attachment [email.hasAttachment]

```
public Attachment viewAttachment(String emailId, String attachmentId) {  
    Email e = viewEmailDetails(emailId);  
    return attachmentService.getAttachment(e, attachmentId);  
}
```

// UC extension: Save/Export Results

```
public byte[] saveOrExportResults(List<Email> emails) {  
    return storageService.exportCsv(emails);  
}
```

```
private void requireUser() {
```

```
    if (currentUser == null) throw new SecurityException("Not authenticated");
```

```
}
```

```
private void requireProject() {  
    if (currentProject == null) throw new IllegalStateException("No project selected");  
}  
}
```

KMSDemo.java

```
import java.time.LocalDate;  
import java.util.HashMap;  
import java.util.List;  
import java.util.Map;  
  
public final class KMSDemo {  
    public static void main(String[] args) {  
  
        // Users  
        Employee emp = new Employee("emp1", "Andrea", "andrea@hk.com", "secret", "PR");  
        ClientUser client = new ClientUser("cli1", "Client A", "client@a.com", "secret", "ClientA  
Ltd");  
  
        Map<String, User> users = new HashMap<>();  
        users.put(emp.getUserId(), emp);  
        users.put(client.getUserId(), client);  
  
        // Project + sample emails  
        Project p = new Project("P100", "Launch", "ClientA", LocalDate.now().minusDays(90));
```

```
Email e1 = new Email(  
    "E1",  
    "Campaign Launch Plan",  
    "Kick-off notes and next steps.",  
    LocalDate.now().minusDays(30),  
    "ted@hk.com",  
    "team@hk.com",  
    null  
);
```

```
Attachment a1 = new Attachment("A1", "budget.xlsx", "application/vnd.ms-excel", 0.3);
```

```
Email e2 = new Email(  
    "E2",  
    "Budget Update Q3",  
    "Revised figures attached.",  
    LocalDate.now().minusDays(20),  
    "cfo@client.com",  
    "andrea@hk.com",  
    List.of(a1)  
);
```

```
p.addEmail(e1);
```

```
p.addEmail(e2);
```

```
// Access map
```



```
Map<String, List<Project>> projectsByUser = new HashMap<>();

projectsByUser.put(emp.getUserId(), List.of(p));
projectsByUser.put(client.getUserId(), List.of(p));


// Services + Facade

KMSSystem kms = new KMSSystem(
    new AuthService(users),
    new ProjectService(projectsByUser),
    new EmailService(),
    new AttachmentService(),
    new StorageService()
);


System.out.println("Auth success? " + kms.authenticateUser("emp1", "secret"));
kms.selectProject("P100");


// Search

List<Email> results = kms.applyFilters(new Filter("budget", null, null, null, false));
System.out.println("Search results: " + results.size());
for (Email r : results) {
    System.out.println("- " + r.getSubject() + " | " + r.preview());
}


// View details + optional attachment

if (!results.isEmpty()) {
    Email opened = kms.viewEmailDetails(results.get(0).getEmailId());
```

```
System.out.println("Opened: " + opened.getSubject() + " | hasAttachment=" +  
opened.hasAttachment());
```

```
if (opened.hasAttachment()) {  
    Attachment att = kms.viewAttachment(  
        opened.getEmailId(),  
        opened.getAttachments().get(0).getAttachmentId()  
    );  
    System.out.println("Downloaded bytes: " + att.download().length);  
}  
}
```

```
// Export
```

```
byte[] csv = kms.saveOrExportResults(results);  
System.out.println("CSV export:\n" + new String(csv));
```

```
// Observer demo
```

```
p.addEmail(new Email(  
    "E3",  
    "New Project Update",  
    "New email added to project.",  
    LocalDate.now(),  
    "lead@hk.com",  
    "team@hk.com",  
    null  
));
```

```
        System.out.println("Notifications: " + emp.getNotifications());
    }
}
```

DomainInvariantTests.java

```
package com.hkunet.kms;

import org.junit.jupiter.api.Test;
import com.hkunet.kms.Email;
import com.hkunet.kms.Attachment;
import com.hkunet.kms.Filter;
import com.hkunet.kms.Project;
import com.hkunet.kms.Employee;

import java.time.LocalDate;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class DomainInvariantTests {

    // ===== EMAIL TESTS =====

    @Test
    void emailWithNullSubjectShouldFail() {
        IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
```

```
() -> new Email(
    "id1",
    null,          // subject (invalid)
    "body",
    LocalDate.now(),
    "sender",
    "receiver",
    List.of()
)

);

assertTrue(
    exception.getMessage().toLowerCase().contains("subject"),
    "Expected exception message to mention 'subject'"
);
}

@Test
void emailWithEmptySubjectShouldFail() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> new Email(
            "id2",
            "",          // subject (empty - invalid)
            "body",
            LocalDate.now(),
```

```
        "sender",
        "receiver",
        List.of()
    )
);

assertTrue(
    exception.getMessage().toLowerCase().contains("subject"),
    "Expected exception message to mention 'subject'"
);
}
```

@Test

```
void emailWithFutureDateShouldFail() {
    LocalDate tomorrow = LocalDate.now().plusDays(1);
```

```
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> new Email(
            "id3",
            "Test Subject",
            "body",
            tomorrow,        // sentAt (future - invalid)
            "sender",
            "receiver",
            List.of()
```

```
    )  
    );  
  
    assertTrue(  
        exception.getMessage().toLowerCase().contains("future") ||  
        exception.getMessage().toLowerCase().contains("date"),  
        "Expected exception message to mention 'future' or 'date'"  
    );  
}
```

@Test

```
void emailWithNullFromShouldFail() {  
    IllegalArgumentException exception = assertThrows(  
        IllegalArgumentException.class,  
        () -> new Email(  
            "id4",  
            "Test Subject",  
            "body",  
            LocalDate.now(),  
            null, // from (invalid)  
            "receiver",  
            List.of()  
        )  
    );  
  
    assertTrue(  

```

```
        exception.getMessage().toLowerCase().contains("from"),
        "Expected exception message to mention 'from'"
    );
}

// ===== ATTACHMENT TESTS =====

@Test
void attachmentWithNullFileNameShouldFail() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> new Attachment(
            "a1",
            null,          // fileName (invalid)
            "application/pdf",
            1.0
        )
    );

    assertTrue(
        exception.getMessage().toLowerCase().contains("filename") ||
        exception.getMessage().toLowerCase().contains("name"),
        "Expected exception message to mention 'filename' or 'name'"
    );
}
```

@Test

```
void attachmentWithNegativeSizeShouldFail() {  
    IllegalArgumentException exception = assertThrows(  
        IllegalArgumentException.class,  
        () -> new Attachment(  
            "a2",  
            "file.pdf",  
            "application/pdf",  
            -1.0          // sizeMb (negative - invalid)  
        )  
    );  
  
    assertTrue(  
        exception.getMessage().toLowerCase().contains("size") ||  
        exception.getMessage().toLowerCase().contains("negative"),  
        "Expected exception message to mention 'size' or 'negative'"  
    );  
}
```

// ===== FILTER TESTS =====

@Test

```
void filterWithInvalidDateRangeShouldFail() {  
    LocalDate startDate = LocalDate.of(2025, 12, 31);  
    LocalDate endDate = LocalDate.of(2025, 1, 1); // End before start (invalid)
```



```
IllegalArgumentException exception = assertThrows(
    IllegalArgumentException.class,
    () -> new Filter(
        "keyword",
        "sender",
        startDate,    // dateFrom
        endDate,      // dateTo (before dateFrom - invalid)
        false
    )
);

assertTrue(
    exception.getMessage().toLowerCase().contains("date") ||
    exception.getMessage().toLowerCase().contains("range"),
    "Expected exception message to mention 'date' or 'range'"
);
}
}
```